

Proyecto Final: Ingeniería Web y Móvil - ICI 4247-01 y ICI 4247 -02

1. Información

- Deben usar para el frontend el framework ionic con react y para el backend node.js, Flask, Django, FastAPI, etc.
- El software debería ser implementado en su totalidad por el grupo de trabajo.
- El problema a resolver deber ser propuesto y modelado por los integrantes del grupo.
- El grupo deberá estar conformado máximo de 4 estudiantes. Los estudiantes deberán inscribir sus proyectos en el siguiente link Formulario de Google, el cual tendrá como fecha **límite 22 de Marzo del 2026**
- El proyecto deberá ser gestionado a través de GitHub, manteniendo un repositorio público con el código fuente del proyecto. También deberán hacer uso de **readme.md**

2. Restricciones

- Los proyectos no se pueden repetir más de 3 veces. En caso de que exista un tema igual o similar, uno de los grupos deberá cambiar su tema, dándose prioridad al grupo que inscribió primero la propuesta.
- El frontend deberá ser desarrollado utilizando el framework Ionic con React.
- El backend deberá ser desarrollado aplicando APIs RESTful con conexión a base de datos relacional, usando Node.js con express o Flask (Python).

- Todos los integrantes del grupo deben pertenecer al mismo paralelo.
- Tanto el repositorio como el proyecto Github deberán ser públicos o en su defecto, deberán proporcionar acceso a la profesor/a y al/la ayudante de la asignatura. Si no es público tendrá una penalización de un punto tomando en escala de 1 a 7.

3. Gestión del Proyecto

El proyecto deberá ser gestionado a través de GitHub, manteniendo un repositorio público con el código fuente del proyecto y manejo correcto de un archivo readme.md. Las funcionalidades que deberán ser usadas **Issues**, **Discussions** y gestionando a través de un tablero de trabajo en **GitHub Projects**.

Los conceptos relacionados al uso de GitHub serán cubiertos durante las ayudantías del curso, y se les hará entrega de material de apoyo accesible a través del Aula Virtual de la asignatura.

4. Entregas del proyecto

El proyecto tendrá dos (2) entregables (Entrega Parcial 1 y Entrega Parcial 2), y una entrega final, la que deberá ser presentada y defendida. **Requisitos** Todas las entregas deben cumplir con los siguientes requisitos.

- Cada entrega debe ser respaldada en GitHub a través de un **commit** el que debe ser subido (push) al repositorio del proyecto **antes de la fecha y hora límite de entrega**.
- Cada entrega debe contar con un **README.md** que contenga la descripción de la entrega, los integrantes y los pasos para ejecutar el proyecto.
- En el Aula Virtual de la asignatura se debe subir un archivo readme.txt que contenga la url del repositorio.

- Se deben incluir otros archivos como diagramas, presentaciones o cualquier otro material que considere necesario para la entrega, dentro de una carpeta llamada **otros** dentro de la raíz del proyecto.
- Se debe mantener el código ordenado, bien documentado, y con los **issues**, **pull request** y **ramas (branches)** bien gestionadas.

5. Entregas Parciales

1. Entrega Parcial 1: Diseño y Estructura inicial

Objetivo: Definir el diseño de la aplicación y construir la estructura del frontend en **Ionic+ React**.

- **EP 1.1:** Definición de al menos 7 requerimientos funcionales y al menos 3 no funcionales (rendimiento, seguridad, usabilidad). Estas funcionalidades no pueden ser repetitivas. Estas funcionalidades están fuera de inicio de sesión o registrarse ya que deben estar inmersas en la propuesta. Dentro de las funcionalidades deben considerar dos tipos de roles (ejemplo: usuario y admin).
- **EP 1.2:** Justificación del problema y análisis del usuario objetivo.
- **EP 1.3:** Bocetos de UI/UX y prototipo en Figma de al menos 7 mockups o pantallas distintas, cada una correspondiente a una funcionalidad previamente definida en los requerimientos del proyecto. Cada pantalla deberá presentar un diseño diferenciado, coherente con el flujo de navegación y la jerarquía de información. Las interfaces deberán ser prototipadas considerando explícitamente: versión móvil y web. El diseño deberá evidenciar distribución de contenido, componentes de navegación (por ejemplo: menú lateral en web, barra inferior en móvil), jerarquía visual y densidad de la información. Se deberá Incluir en los mockups dos formularios relacionados al inicio de sesión de usuarios y registro, considerando los campos: Nombre de usuario, RUT, Correo Electrónico, Región, Comuna, Contraseña, Confirmación de Contraseña y aceptación de términos y condiciones. Considerando validaciones visuales y diseño centrado en el usuario.

- **EP 1.4:** Definición de Arquitectura de Navegación y Experiencia del Usuario. El equipo deberá definir la arquitectura de navegación de la aplicación, describiendo la estructura de rutas, jerarquía de vistas, y flujo de interacción entre pantallas. La entrega deberá incluir: (a) Rutas principales y secundarias; (b) Relaciones jerárquicas entre vistas; (c) Flujo de navegación entre funcionalidades; (d) diferenciación de acceso según roles (por ejemplo: usuario /administrador); (e) flujo de principales tareas (task flow), (f) puntos críticos de interacción; (g) coherencia de experiencia entre dispositivos; (h) breve justificación técnica de las decisiones adoptadas, considerando usabilidad, eficiencia de interacción, claridad estructural y escalabilidad de la arquitectura frontend.
- **EP 1.5:** Creación del proyecto en Ionic con React, considerando: (a) Uso de react router; (b) Rutas públicas y rutas protegidas; (c) Redirecciones (ejemplo: login obligatorio); (d) Estructura modular de vistas.
- **EP 1.6:** Diseño de pantallas principales e incorporando una estructura de navegación funcional y coherente con la arquitectura previamente definida en ionic-react (al menos 4). Uso de componentes propios de Ionic (IonPage, IonHeader, IonContent, IonTabs, IonMenu, etc). Separación estructural del código en carpetas (pages, components, routes, services).

Entrega:

- Repositorio GitHub/GitLab con el código del frontend
- Prototipo UI/UX en Figma.
- Aplicación navegable con diseño básico realizado en Ionic + React.
- Readme.md que incluya la documentación técnica.

2. Entrega Parcial 2: Integración frontend + backend y Autenticación

Objetivo: Implementar el backend en Node.js con Express, Flask u otro framework backend, y conectarlo con el frontend y agregar autenticación.

- **EP 2.1:** Creación del servidor en Node.js con Express o Flask

- **EP 2.2:** Configuración y modelado de la base de datos relacional. Deberán incluir el diseño modelo relacional y la implementación en base de datos usando PostgreSQL/ MySQL, uso de ORM (opcional) y validación de integridad.
- **EP 2.3:** Desarrollo de API REST con endpoints básicos (GET, POST, PUT/PATCH, y DELETE). Manejo adecuado de códigos HTTP y respuestas en formato JSON estructurado.
- **EP 2.4:** Consumo de la API REST desde Ionic con React utilizando fetch o Axios, implementando manejo de errores, interceptores y gestión de tokens JWT.
- **EP 2.5:** Implementación de autenticación con JWT deberá incluir (a) formulario de registro e inicio de sesión; (b) rutas protegidas en frontend; (c) generación y validación de JWT; (d) diferenciación por roles.
- **EP 2.6:** Validación de usuarios y manejo de sesiones deberá incluir: (a) validación de inputs ; (b) Hash de contraseñas con bcrypt; (c) Manejo seguro de credenciales; (d) Protección básica contra inyección SQL.
- **EP 2.7:** Pruebas funcionales, considerando (a) Pruebas en Postman o Insomnia; (b) Documentación de endpoints; (c) Evidencia de pruebas.

Entrega:

- Backend funcional con API REST
- Autenticación y manejo de usuarios
- Integración frontend + backend con pruebas en Postman o Insomnia.
- Repositorio actualizado o crear un nuevo repositorio para backend y otro para front-end.

3. **Entrega Final:** Funcionalidades avanzadas, optimizar rendimiento y preparar aplicación para su despliegue.

Objetivo: Agregar funcionalidades avanzadas, optimizar rendimiento y preparar la aplicación para su despliegue.

- **EF 1:** Implementación de funcionalidades completas (CRUD, notificaciones, almacenamiento local) de acuerdo a los requerimientos funcionales y no funcionales definidos.
- **EF 2:** Mejoras en UI/UX y optimización de rendimiento.
- **EF 3:** Implementación de seguridad avanzada en API: protección contra inyección SQL/XSS, implementación CORS seguro, y encriptación de datos sensibles con bcrypt.
- **EF 4:** Optimización de consultas y respuesta eficiente.
- **EF 5:** Integración con algún servicio externo (AWS o APIs de terceros).
- **EF 6:** Despliegue con docker
 - Creación de Dockerfile para backend y frontend y docker compose.
 - Configuración de docker-compose para orquestación de servicios (API, base de datos, frontend).
 - Pruebas de despliegue local con Docker.

Entrega:

- Aplicación con CRUD completo, UI/UX optimizado y seguridad web aplicada.
- Backend seguro con validaciones y cifrado de datos.
- Documentación técnica del proyecto.

6. Tecnologías y Herramientas

Frontend:

- Ionic, React, TypeScript.
- Ionic Native y Capacitor.
- CSS con Tailwind o Bootstrap.

Backend:

- Node.js con Express.js
- Flask con Python
- Bases de datos : PostgreSQL, MongoDB o MySQL
- Autenticación: JWT, OAuth.
- Cloud Services: AWS, Firebase.

Desarrollo y Despliegue:

- Git y Github/ GitLab
- Postman o Insomnia para pruebas de API
- Docker para entornos de desarrollo.

7. Distribución general de puntajes

Criterio de evaluación	Puntaje
EF1. Funcionalidades completas e integración funcional de la aplicación	20 pts
EF2. Mejoras de UI/UX y optimización del rendimiento	15 pts
EF3. Seguridad avanzada en la API y manejo seguro de datos	15 pts
EF4. Optimización de consultas y eficiencia de respuesta	10 pts
EF5. Integración con servicio externo o API de terceros	10 pts
EF6. Despliegue local mediante Docker y docker-compose	15 pts
Presentación y defensa del proyecto	15 pts
Total	100 pts

Cuadro 1: Distribución de puntajes de la Entrega Final.

8. Rúbrica analítica

Cuadro 2: Rúbrica analítica para la evaluación de la Entrega Final.

Criterio	Puntaje	Excelente	Bueno	Básico	Insuficiente
EF1. Funcionalidades completas e integración funcional de la aplicación	20 pts	Implementa de forma completa y funcional los requerimientos definidos. Incluye CRUD completo, autenticación, diferenciación por roles, notificaciones, almacenamiento local y flujo integrado entre frontend y backend. La aplicación es estable y ejecutable.	Implementa la mayoría de las funcionalidades principales. El CRUD funciona en la mayor parte de los casos y existe integración frontend-backend, aunque presenta errores menores o funcionalidades secundarias incompletas.	Implementa parcialmente las funcionalidades. El CRUD o la integración presentan fallas relevantes. Algunas funciones dependen de datos simulados, están incompletas o no se relacionan claramente con los requerimientos originales.	No implementa las funcionalidades centrales, la aplicación no es ejecutable o no existe integración real entre frontend y backend.
EF2. Mejoras de UI/UX y optimización del rendimiento	15 pts	Presenta una interfaz clara, consistente, responsiva y coherente entre versión móvil y web. Usa adecuadamente componentes Ionic, navegación fluida, validaciones visuales y tiempos de carga adecuados.	La interfaz es funcional y mayormente consistente. Existen mejoras de diseño y navegación, aunque pueden persistir problemas menores de responsividad, densidad visual o rendimiento.	La interfaz permite usar la aplicación, pero presenta problemas evidentes de usabilidad, navegación, coherencia visual o rendimiento. El diseño no siempre respeta el flujo definido.	La interfaz es confusa, incompleta o difícil de usar. No evidencia mejoras respecto a entregas anteriores o presenta fallas graves de visualización y rendimiento.
EF3. Seguridad avanzada en la API y manejo seguro de datos	15 pts	Implementa validaciones robustas, protección contra inyección SQL y XSS, CORS configurado de forma segura, manejo correcto de JWT, rutas protegidas, hash de contraseñas con bcrypt y manejo seguro de credenciales.	Implementa mecanismos de seguridad principales, incluyendo JWT, bcrypt y validaciones. Presenta detalles menores pendientes en CORS, sanitización de entradas o control de errores.	Incluye seguridad básica, pero incompleta. Hay validaciones parciales, manejo débil de errores, configuración insegura o protección insuficiente frente a entradas maliciosas.	No implementa medidas de seguridad relevantes, almacena credenciales de forma insegura o permite acceso no autorizado a rutas o datos sensibles.
EF4. Optimización de consultas y eficiencia de respuesta	10 pts	Las consultas a la base de datos están bien estructuradas, usan filtros, relaciones e índices cuando corresponde. Las respuestas son eficientes, consistentes y no exponen datos innecesarios.	Las consultas funcionan correctamente y presentan una eficiencia aceptable. Puede existir duplicación menor de lógica o respuestas con información adicional no crítica.	Las consultas funcionan parcialmente, pero son poco eficientes, repetitivas o no consideran adecuadamente filtros, relaciones o manejo de errores.	Las consultas fallan, son inseguras, generan respuestas incorrectas o impiden el funcionamiento estable de la aplicación.
EF5. Integración con servicio externo o API de terceros	10 pts	Integra correctamente un servicio externo pertinente al problema. La integración se justifica, se documenta, maneja errores, claves o variables de entorno y aporta valor real a la aplicación.	Integra un servicio externo funcional y relacionado con el proyecto, aunque con documentación limitada o manejo de errores parcial.	La integración es superficial, poco justificada o funciona de manera limitada. Puede depender de datos estáticos o no aportar valor claro al flujo principal.	No existe integración con servicio externo o la integración no funciona.

∞

Criterio	Puntaje	Excelente	Bueno	Básico	Insuficiente
EF6. Despliegue local mediante Docker y docker-compose	15 pts	Incluye Dockerfile para frontend y backend, docker-compose para orquestar frontend, API y base de datos, variables de entorno, instrucciones claras y evidencia de despliegue local exitoso.	Incluye configuración Docker funcional para la mayoría de los servicios. Puede requerir ajustes menores o instrucciones adicionales para ejecutar correctamente.	La configuración Docker es parcial. Algunos servicios no se levantan correctamente o requieren pasos manuales no documentados.	No incluye Docker funcional, no existe docker-compose o el proyecto no puede ejecutarse mediante contenedores.
Presentación y defensa del proyecto	15 pts	El equipo presenta con claridad el problema, solución, arquitectura, funcionalidades, decisiones técnicas, pruebas y despliegue. Todos los integrantes participan y responden adecuadamente preguntas técnicas.	La presentación comunica los aspectos principales del proyecto, aunque con menor profundidad en decisiones técnicas, pruebas o despliegue. La mayoría del equipo participa.	La presentación es poco clara o incompleta. Se explican algunas funcionalidades, pero no se justifica adecuadamente la arquitectura ni las decisiones técnicas.	No se presenta el proyecto, la defensa no evidencia dominio técnico o no se logra demostrar el funcionamiento de la aplicación.